MODULE M203 — BASES DE DONNÉES AVANCÉES • ATELIER PRATIQUE N° 3

Atelier 3 - Gestion d'un Centre de Formation avec Procédures et Fonctions Stockées

Réalisé par : Basma Elmaimouni **Environnement :** MySQL 8.4.11 (Aiven Cloud & Client CLI)

Objectif global et Contexte

Objectif global : Concevoir et manipuler une base de données relationnelle MySQL permettant de gérer efficacement les formateurs, les stagiaires et les sessions d'un centre de formation. Cet atelier vise la maîtrise de la programmation côté serveur SGBD à travers l'implémentation de procédures stockées, de fonctions déterministes, de structures de contrôle (boucles `WHILE`, conditions `IF...ELSE`) ainsi que la gestion des modes de passage de paramètres `IN`, `OUT` et `INOUT`.

Contexte : Le centre de formation **TechLearn** propose plusieurs sessions animées par des formateurs spécialisés. Chaque session est caractérisée par un formateur référent, une capacité maximale d'accueil et un effectif d'inscrits. Afin d'optimiser la gestion administrative, l'établissement souhaite automatiser quatre opérations clés : le calcul dynamique du taux d'occupation des sessions, la mise à jour contrôlée du nombre d'inscrits, la purge des sessions vides et la génération automatisée d'un calendrier de sessions.

Environnement de déploiement Cloud (Aiven MySQL)

Pour l'exécution de cet atelier, une instance managée **MySQL 8.4.11** (service `mysql-16755a2b`) a été déployée sur la plateforme cloud Aiven et administrée en ligne de commande via le client officiel MySQL avec chiffrement SSL (`--ssl-mode=REQUIRED`).

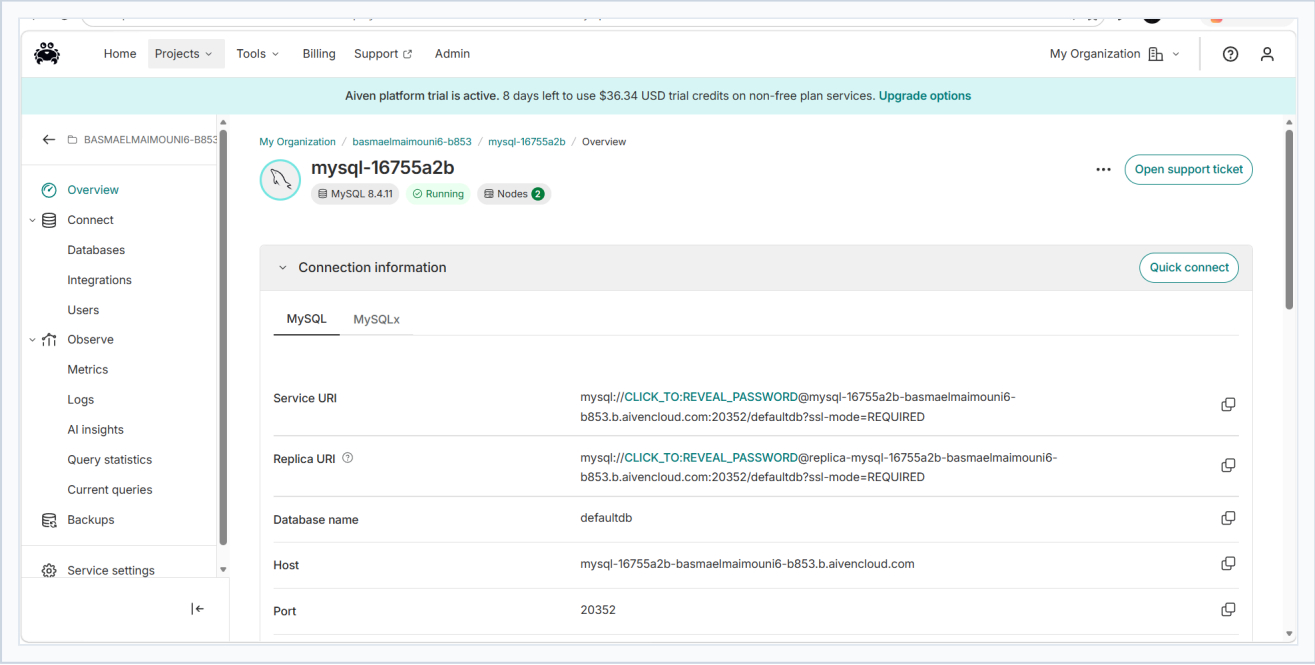


Figure 1 : Configuration de l'instance MySQL 8.4.11 sur Aiven Cloud (hôte, port 20352 et base defaultdb).

1. Création de la base et des tables

Après connexion au serveur distant, la base `techlearn_db` a été créée et la structure relationnelle composée des trois tables `formateurs`, `stagiaires` et `sessions` (reliée par clé étrangère à `formateurs`) a été implémentée dans la base active.

```
CREATE DATABASE techlearn_db;
USE techlearn_db;

CREATE TABLE formateurs (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(100),
    specialite VARCHAR(100)
);

CREATE TABLE stagiaires (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(100),
    ville VARCHAR(100)
);

CREATE TABLE sessions (
    id INT AUTO_INCREMENT PRIMARY KEY,
    intitule VARCHAR(100),
    formateur_id INT,
    capacite INT,
    inscrits INT DEFAULT 0,
    FOREIGN KEY (formateur_id) REFERENCES formateurs(id)
);
```

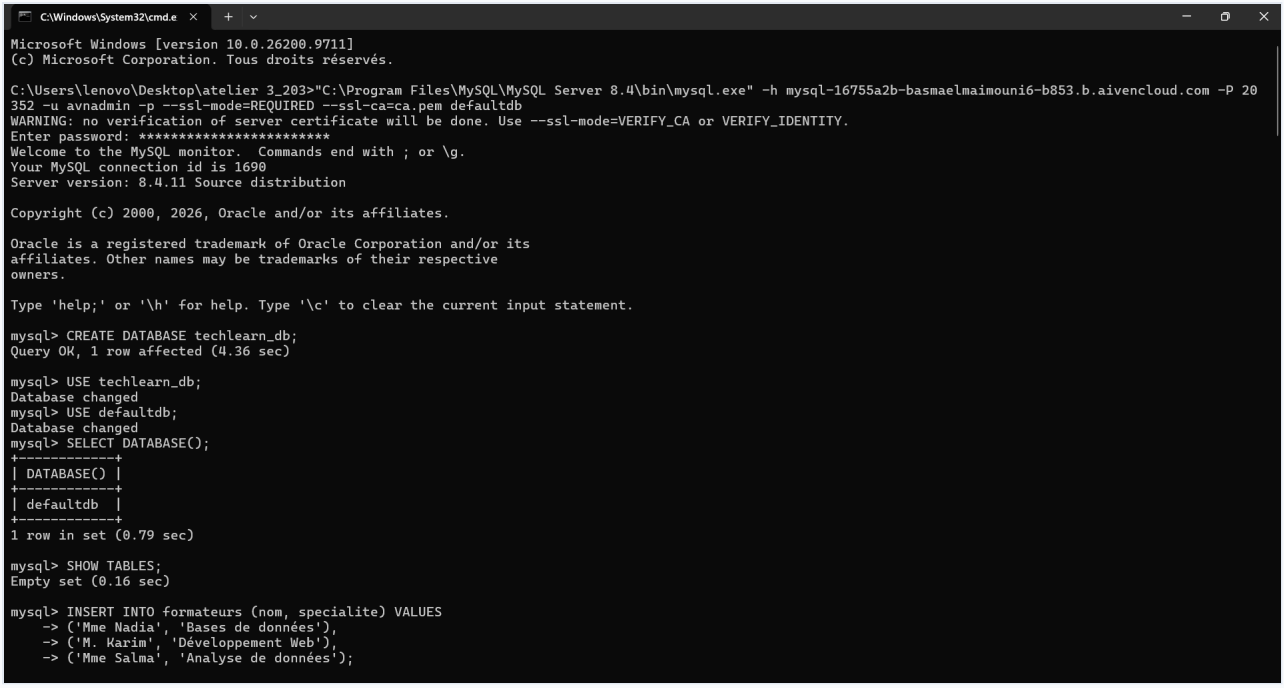


Figure 2 : Connexion sécurisée au serveur MySQL 8.4.11 et initialisation de la base de données.

2. Insertion de données

Les trois tables ont ensuite été alimentées avec le jeu d'essai initial comprenant trois formateurs spécialisés, trois stagiaires répartis sur Casablanca, Rabat et Marrakech, ainsi que trois sessions de formation.

```
INSERT INTO formateurs (nom, specialite) VALUES
('Mme Nadia', 'Bases de données'),
('M. Karim', 'Développement Web'),
('Mme Salma', 'Analyse de données');

INSERT INTO stagiaires (nom, ville) VALUES
('Ali', 'Casablanca'),
('Sara', 'Rabat'),
('Mouna', 'Marrakech');

INSERT INTO sessions (intitule, formateur_id, capacite, inscrits) VALUES
('MySQL avancé', 1, 20, 5),
('HTML/CSS', 2, 25, 10),
('Python Data', 3, 15, 8);
```

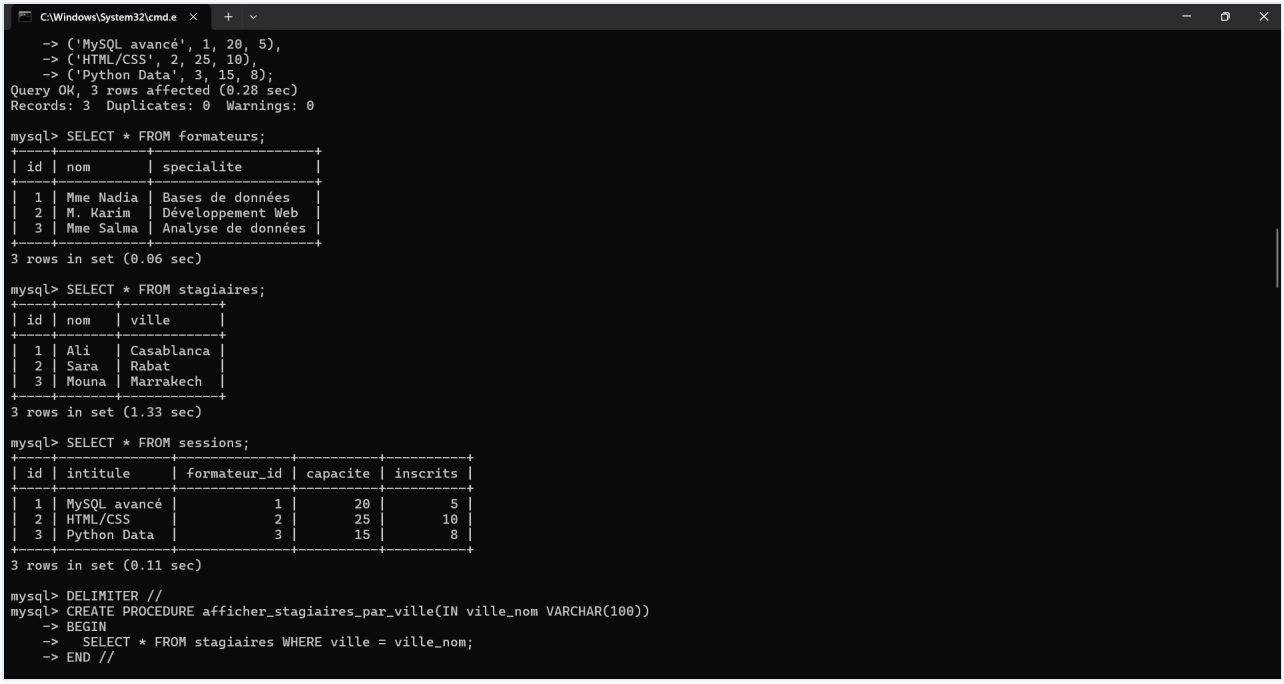


*Figure 3 : Vérification par requêtes SELECT * des enregistrements insérés dans formateurs, stagiaires et sessions.*

3. Travail demandé

Exercice 1 - Procédure avec paramètre IN

Objectif : Afficher la liste des stagiaires résidant dans une ville passée en paramètre d'entrée (`IN`).

```
DELIMITER //
CREATE PROCEDURE afficher_stagiaires_par_ville(IN ville_nom VARCHAR(100))
BEGIN
    SELECT * FROM stagiaires WHERE ville = ville_nom;
END //
DELIMITER ;

CALL afficher_stagiaires_par_ville('Casablanca');
```

> **Analyse des résultats :** L'appel avec l'argument `'Casablanca'` filtre dynamiquement la table `stagiaires` et retourne uniquement l'enregistrement correspondant à *Ali* (id = 1). Le paramètre `IN` transmet une valeur en lecture seule au bloc procédural.

Exercice 2 - Procédure avec paramètre OUT

Objectif : Calculer et retourner le nombre total de sessions enregistrées via un paramètre de sortie (`OUT`).

```
DELIMITER //
CREATE PROCEDURE compter_sessions(OUT total INT)
BEGIN
    SELECT COUNT(*) INTO total FROM sessions;
END //
DELIMITER ;

CALL compter_sessions(@nb);
SELECT @nb AS nombre_total_sessions;
```

Analyse des résultats : La variable de session @nb reçoit le résultat de l'agrégation `COUNT(*)` affecté au paramètre `OUT total`. L'interrogation ultérieure de @nb affiche la valeur **3**, correspondant aux trois sessions initiales.

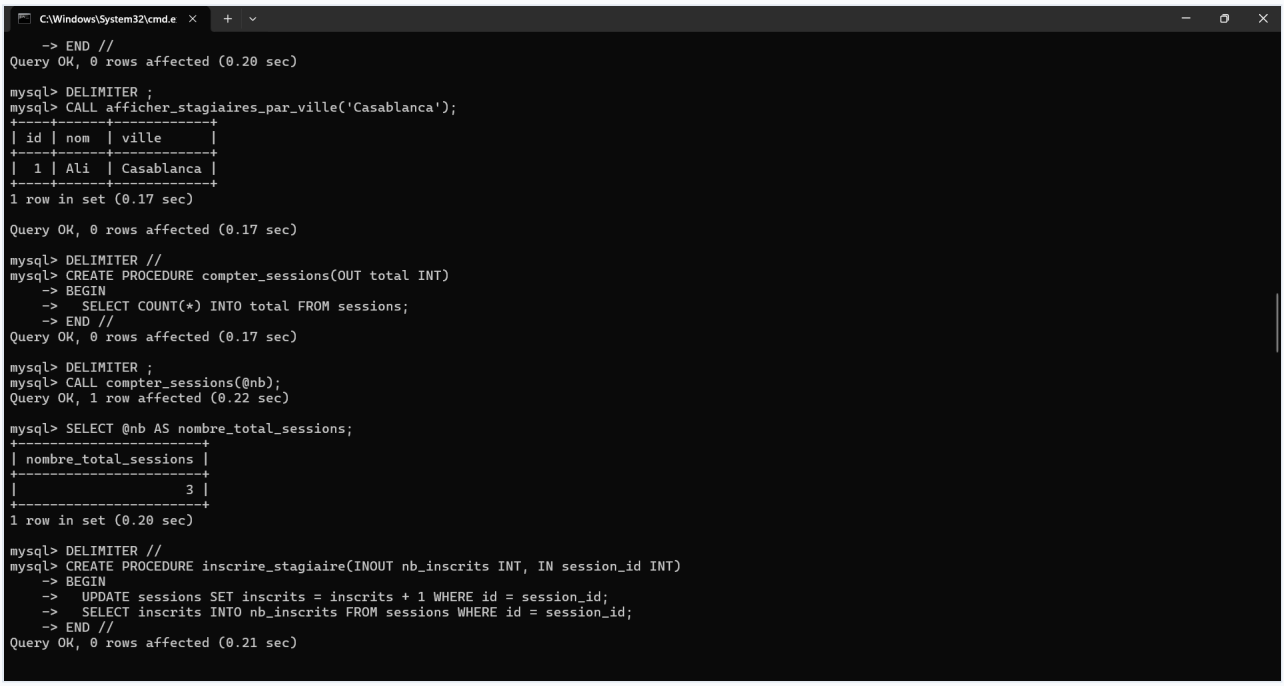


Figure 4 : Résultats d'exécution des exercices 1 (paramètre IN) et 2 (paramètre OUT).

Exercice 3 - Procédure avec paramètre INOUT

Objectif : Incrémenter le nombre d'inscrits d'une session ciblée et renvoyer le nouvel effectif à travers le même paramètre bidirectionnel (`INOUT`).

```
DELIMITER //
CREATE PROCEDURE inscrire_stagiaire(INOUT nb_inscrits INT, IN session_id INT)
BEGIN
    UPDATE sessions SET inscrits = inscrits + 1 WHERE id = session_id;
    SELECT inscrits INTO nb_inscrits FROM sessions WHERE id = session_id;
END //
DELIMITER ;

SET @nouveau = 0;
CALL inscrire_stagiaire(@nouveau, 1);
SELECT @nouveau AS nouveau_nombre_inscrits;
```

Analyse des résultats : Initialisée à 0, la variable `@nouveau` est passée en mode `INOUT` pour la session 1 (*MySQL avancé*, qui comptait 5 inscrits). La procédure incrémente la colonne `inscrits` en base et réinjecte la nouvelle valeur **6** dans `@nouveau`.

Exercice 4 - Fonction : taux d'occupation d'une session

Objectif : Créer une fonction scalaire retournant le pourcentage de remplissage d'une session donnée.

```
DELIMITER //
CREATE FUNCTION taux_occupation(id_session INT)
RETURNS DECIMAL(5,2)
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE taux DECIMAL(5,2);
    SELECT (inscrits / capacite) * 100 INTO taux FROM sessions WHERE id =
id_session;
    RETURN taux;
END //
DELIMITER ;

SELECT intitule, taux_occupation(id) AS taux FROM sessions;
```

Analyse des résultats : Contrairement à une procédure, la fonction `taux_occupation()` s'intègre directement dans une clause `SELECT`. Elle affiche les taux exacts : **30.00 %** pour *MySQL avancé* (6/20), **40.00 %** pour *HTML/CSS* (10/25) et **53.33 %** pour *Python Data* (8/15).

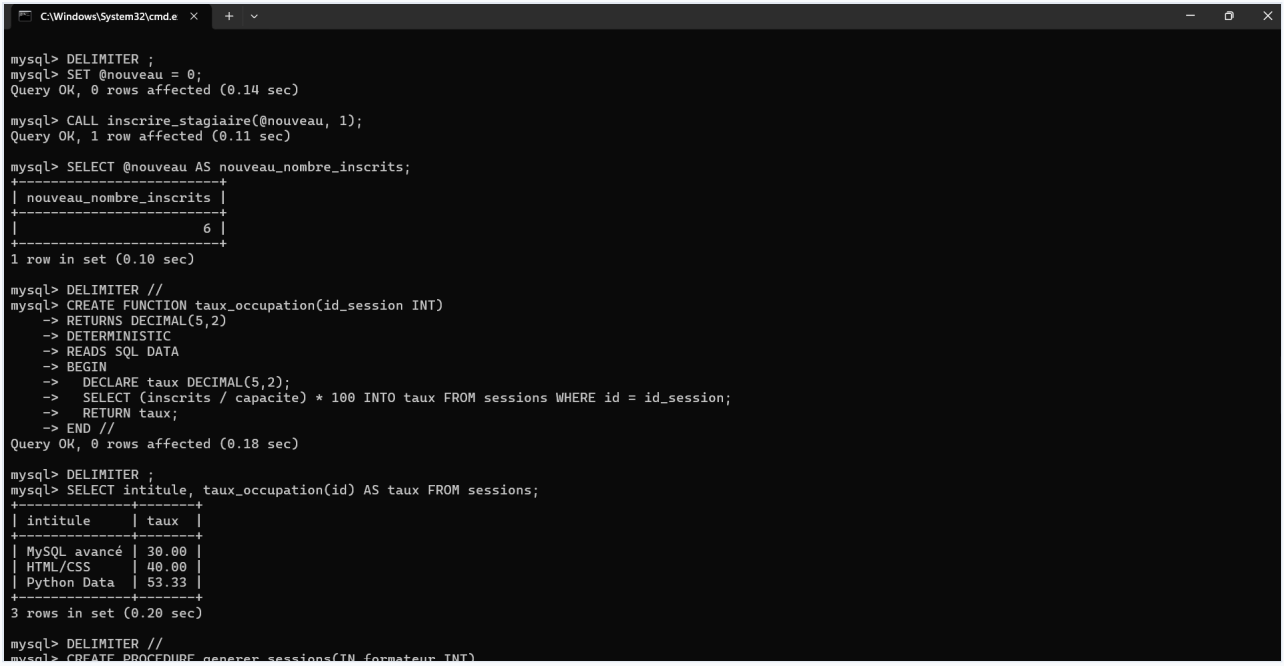


Figure 5 : Démonstration du paramètre INOUT (Exercice 3) et calcul des taux d'occupation par fonction (Exercice 4).

Exercice 5 - Boucle et automatisation

Objectif : Générer automatiquement 10 sessions de 20 places (avec 0 inscrit) attribuées à un formateur spécifique à l'aide d'une boucle `WHILE`.

```
DELIMITER //
CREATE PROCEDURE generer_sessions(IN formateur INT)
BEGIN
    DECLARE i INT DEFAULT 1;
    WHILE i <= 10 DO
        INSERT INTO sessions (intitule, formateur_id, capacite, inscrits)
        VALUES (CONCAT('Session ', i, ' - Formateur ', formateur), formateur, 20,
0);
        SET i = i + 1;
    END WHILE;
END //
DELIMITER ;

CALL generer_sessions(2);
```

> **Analyse des résultats :** L'appel `CALL generer_sessions(2)` exécute 10 itérations successives et insère les lignes d'ID 4 à 13 intitulées *« Session 1 - Formateur 2 »* jusqu'à *« Session 10 - Formateur 2 »*, portant le total à 13 sessions.

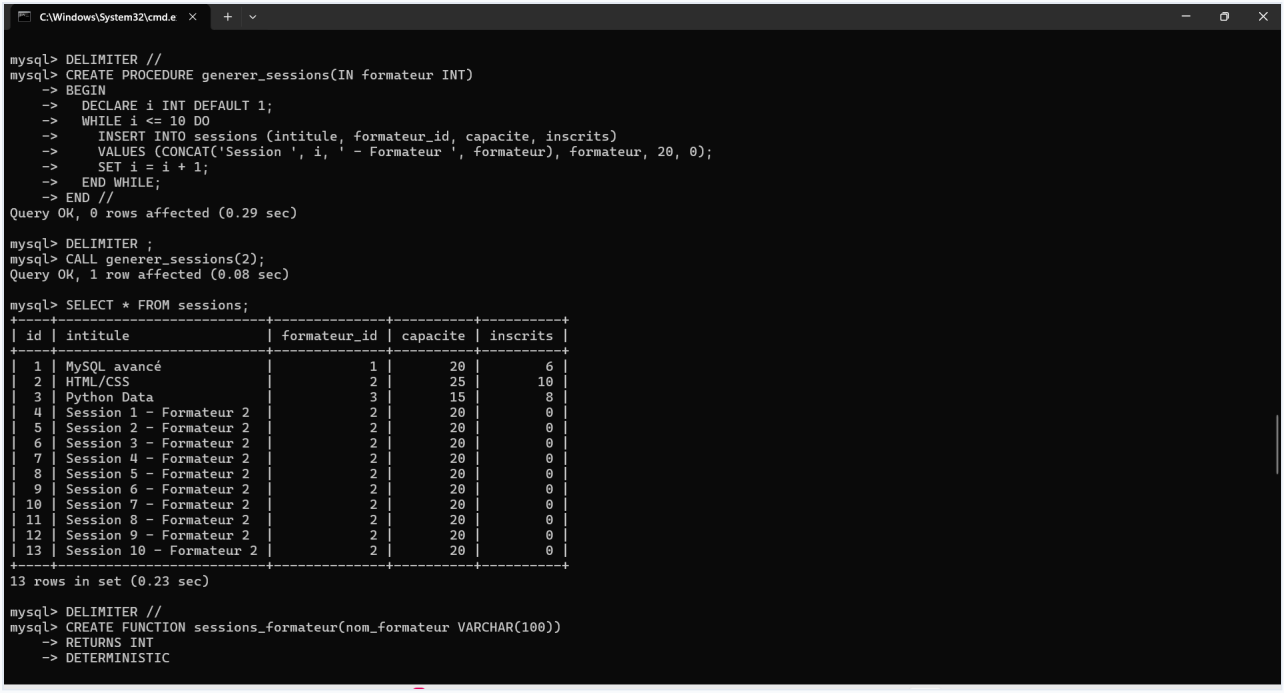


Figure 6 : Création de la procédure generer_sessions avec boucle WHILE et visualisation des 13 sessions résultantes.

4. Défis finaux

Défi 1 - Fonction sessions_formateur(nom_formateur)

Cette fonction réalise une jointure interne entre `sessions` et `formateurs` afin de dénombrer les sessions animées par un formateur identifié par son nom.

```
DELIMITER //
CREATE FUNCTION sessions_formateur(nom_formateur VARCHAR(100))
RETURNS INT
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE nb INT;
    SELECT COUNT(*) INTO nb
    FROM sessions s
    JOIN formateurs f ON s.formateur_id = f.id
    WHERE f.nom = nom_formateur;
    RETURN nb;
END //
DELIMITER ;

SELECT sessions_formateur('M. Karim') AS nb_sessions;
```

> **Analyse des résultats :** L'appel pour `'M. Karim'` (formateur ID 2) retourne **11** sessions, cumulant sa session initiale (*HTML/CSS*) et les 10 sessions générées automatiquement à l'exercice 5.

Défi 2 - Procédure supprimer_sessions_vides()

Cette procédure nettoie la table `sessions` en supprimant tous les enregistrements dont l'effectif d'inscrits est nul.

```
DELIMITER //
CREATE PROCEDURE supprimer_sessions_vides()
BEGIN
    DELETE FROM sessions WHERE inscrits = 0;
END //
DELIMITER ;

CALL supprimer_sessions_vides();
```

> **Analyse des résultats :** L'exécution indique *« Query OK, 10 rows affected »*. Les 10 sessions générées précédemment avec `inscrits = 0` sont supprimées, ne conservant que les 3 sessions actives.

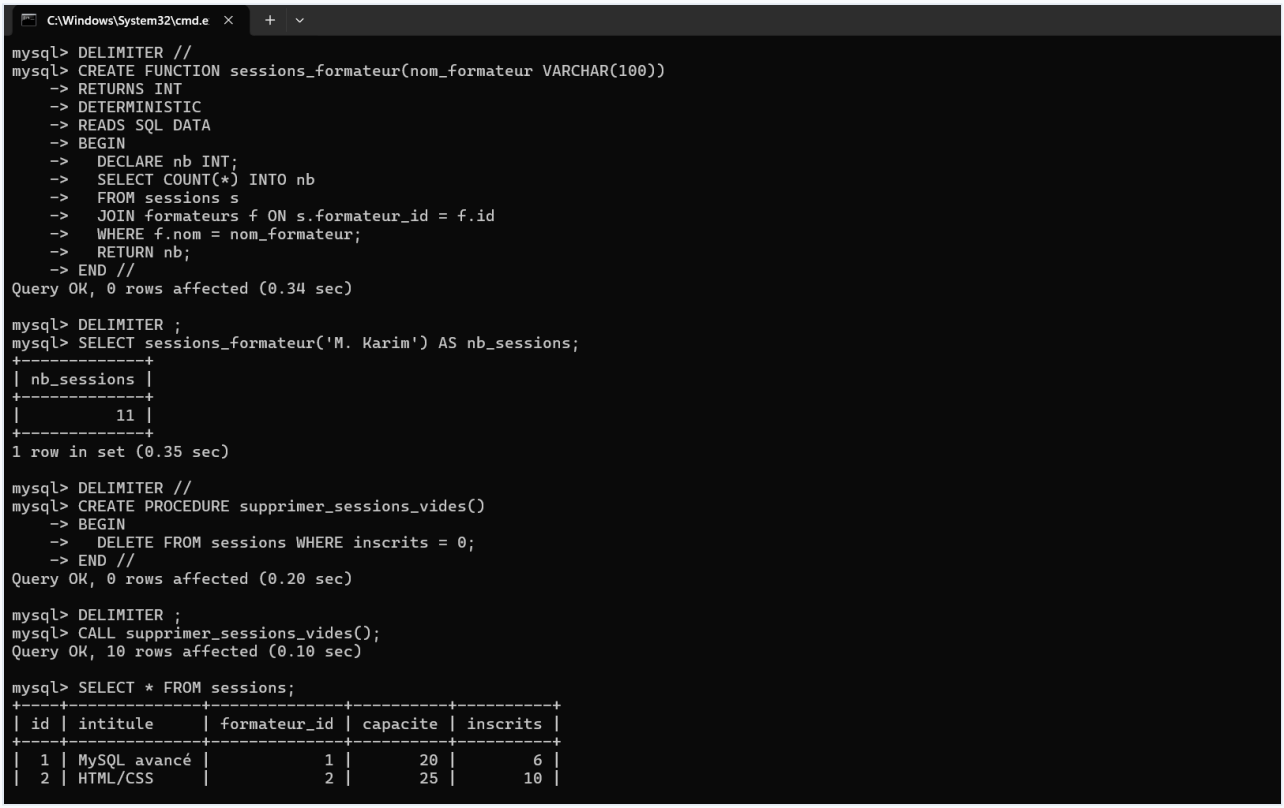


Figure 7 : Comptage des sessions de M. Karim (11) et purge des 10 sessions vides.

Défi 3 - Procédure valider_inscription(IN session_id INT)

Cette procédure implémente un contrôle d'intégrité métier par structure conditionnelle `IF...THEN...ELSE` : elle vérifie si le plafond de capacité est atteint avant d'autoriser une nouvelle inscription.

```
DELIMITER //
CREATE PROCEDURE valider_inscription(IN session_id INT)
BEGIN
    DECLARE nb_inscrits INT;
    DECLARE cap INT;

    SELECT inscrits, capacite INTO nb_inscrits, cap
    FROM sessions WHERE id = session_id;

    IF nb_inscrits >= cap THEN
        SELECT 'Session complète' AS message;
    ELSE
        UPDATE sessions SET inscrits = inscrits + 1 WHERE id = session_id;
        SELECT 'Inscription validée' AS message;
    END IF;
END //
DELIMITER ;

CALL valider_inscription(1);
UPDATE sessions SET inscrits = capacite WHERE id = 3;
CALL valider_inscription(3);
```

Analyse des résultats : Deux scénarios ont été testés : sur la session 1 (6 inscrits pour 20 places), la procédure incrémente l'effectif et affiche **« Inscription validée »**. Après saturation volontaire de la session 3 (`inscrits = capacite`), l'appel bloque la mise à jour et retourne **« Session complète »**.

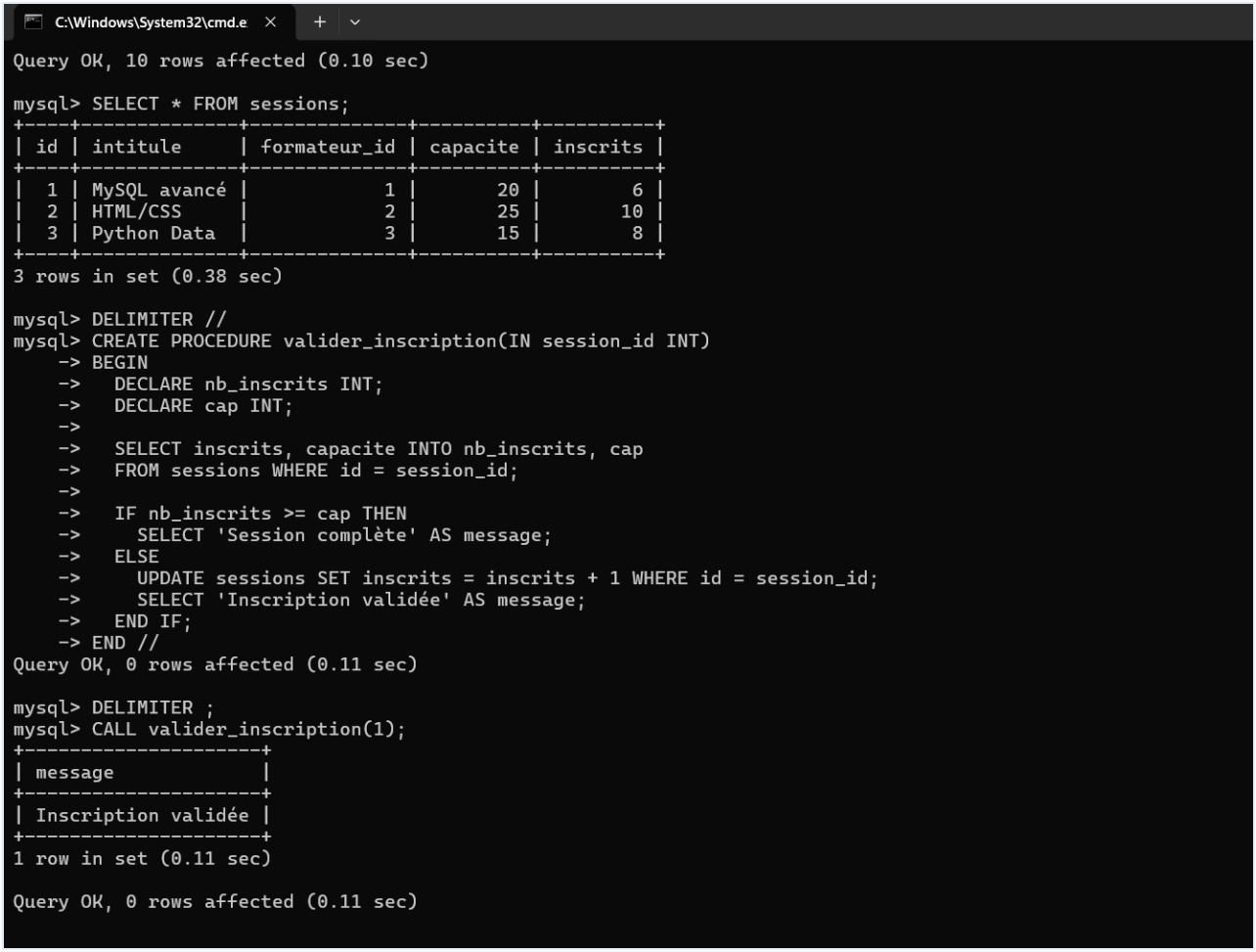


Figure 8 : Implémentation de la procédure valider_inscription et validation d'une inscription sur la session 1.

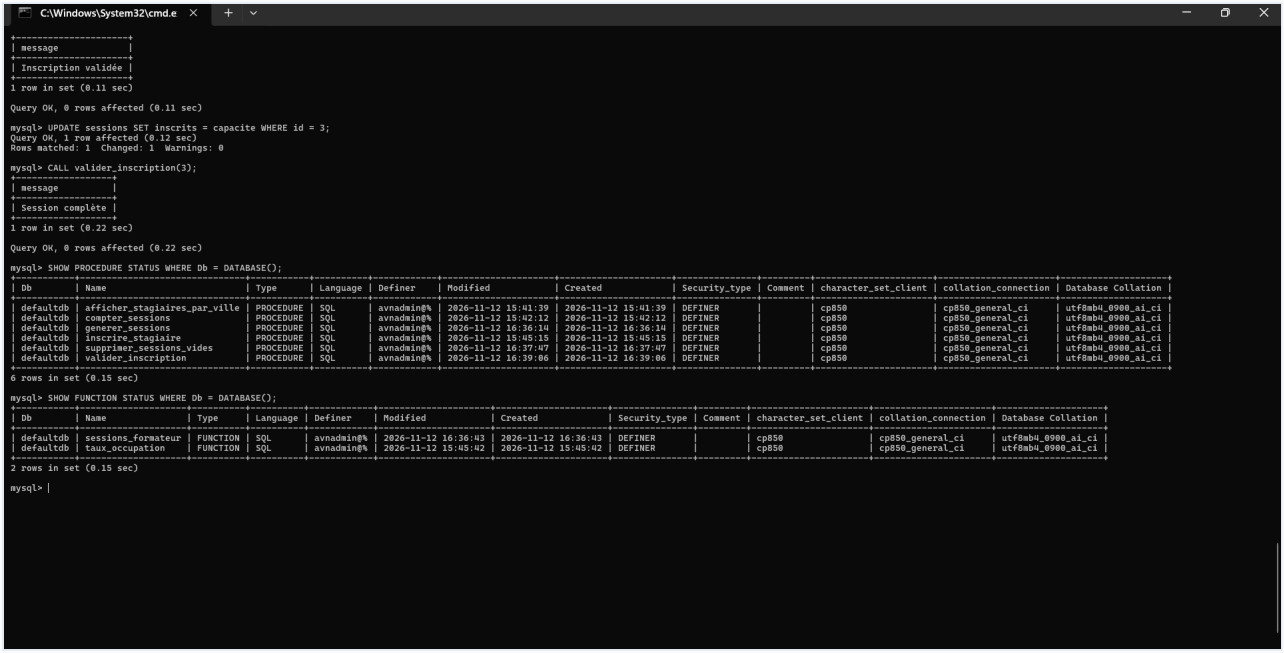


Figure 9 : Blocage sur session complète (session 3) et inventaire complet des 6 procédures et 2 fonctions via SHOW STATUS.

5. Conclusion personnelle

La réalisation de cet atelier sur une instance MySQL 8.4 hébergée sur Aiven Cloud m'a permis de concrétiser l'intérêt de déporter la logique métier directement au sein du système de gestion de bases de données. L'encapsulation des règles de gestion dans des procédures et fonctions stockées garantit une meilleure cohérence des données (notamment lors du contrôle des capacités d'accueil), réduit les échanges réseau entre l'application cliente et le serveur, et simplifie les opérations répétitives grâce aux structures itératives et conditionnelles.

6. Publication du rapport sur GitLab Pages ou Vercel

Conformément aux consignes de diffusion technique, ce rapport finalisé est structuré et exporté au format PDF afin d'être versionné dans un dépôt Git et publié en ligne de manière statique via **GitLab Pages** ou **Vercel**, assurant ainsi une consultation accessible et pérenne des livrables du module M203.

7. Compétences validées

Compétence	Description
Création de procédures/fonctions	Maîtriser les blocs SQL stockés (`CREATE PROCEDURE`, `CREATE FUNCTION`, gestion du `DELIMITER`)
Utilisation de paramètres IN/OUT/INOUT	Comprendre le passage de valeurs en entrée, en sortie et bidirectionnel avec variables de session
Structures de contrôle	Utiliser les branchements conditionnels `IF...ELSE` et les boucles itératives `WHILE`
Organisation et documentation	Structurer un rapport technique complet illustré par des preuves d'exécution et des analyses
Hébergement sur GitLab Pages ou Vercel	Publication de résultats techniques en ligne